





De JavaScript a TypeScript: todo lo que necesitas saber

Nivel: Intermedio JS

Curva de aprendizaje: Rapida

Mayo 2026

Tabla de Contenidos

1. ¿Qué es TypeScript?
2. ¿Por qué usar TypeScript?
 - Problemas reales que resuelve
 - Comparación directa JS vs TS
3. Instalación y configuración
 - tsconfig.json explicado
4. Tipos básicos
 - Primitivos
 - Arrays y Tuplas
 - Any, Unknown, Never
 - Uniones y Literales
5. Funciones en TypeScript
 - Parámetros opcionales y por defecto
 - Sobrecarga de funciones
6. Interfaces
 - Propiedades opcionales y readonly
 - Extender interfaces
 - Interfaces vs Types
7. Clases
 - Modificadores de acceso
 - Herencia
 - Clases abstractas
8. Genéricos
9. Flujo de trabajo recomendado
10. Recursos y siguientes pasos

1. ¿Qué es TypeScript?

TypeScript es un lenguaje de programación de código abierto desarrollado por Microsoft. Es un **superconjunto tipado de JavaScript**, lo que significa que todo código JavaScript válido también es TypeScript válido — pero TypeScript agrega un sistema de tipos estáticos opcional que se verifica en tiempo de compilación, antes de que el código llegue al navegador o servidor.

JavaScript (ES6+)

+ Sistema de Tipos

= TypeScript

TypeScript **no se ejecuta directamente** en el navegador. Se compila (transpila) a JavaScript puro, por lo que es completamente compatible con cualquier entorno que ejecute JS: navegadores, Node.js, Deno, etc. El proceso es simple:

archivo.ts

--> tsc -->

archivo.js

--> Node / Browser

■ TypeScript fue lanzado en 2012 y hoy es uno de los lenguajes más populares del mundo. Angular, NestJS, Next.js y VS Code están escritos completamente en TypeScript.

2. ¿Por qué usar TypeScript?

Si ya sabes JavaScript, quizás te preguntas: *¿para qué agregar tipos si JS funciona bien?* La respuesta está en los problemas que aparecen cuando tu proyecto crece.

2.1 Problemas reales que resuelve

Problema 1	Errores silenciosos en tiempo de ejecución	JS acepta pasar un string donde se espera un numero sin avisar. TS lo detecta al compilar.
Problema 2	Documentación automática	Los tipos actúan como documentación viva: sabes exactamente que acepta y retorna cada función.
Problema 3	Autocompletado inteligente	VS Code y otros editores usan los tipos para ofrecerte sugerencias precisas mientras escribes.
Problema 4	Refactorización segura	Al renombrar una función o cambiar su firma, TS te muestra todos los lugares afectados.
Problema 5	Escalabilidad en equipos	En proyectos grandes con varios developers, los tipos evitan malentendidos sobre la forma de los datos.

2.2 Comparación directa: JS vs TS

El siguiente ejemplo muestra un bug clásico de JavaScript que TypeScript detecta antes de ejecutar el código:

JavaScript	TypeScript
// JavaScript – sin tipos	// TypeScript – con tipos
function sumar(a, b) {	function sumar(
return a + b;	a: number,
}	b: number
	): number {
// Bug silencioso:	return a + b;
sumar(5, "3"); // "53" no 8!	}

// JS no avisa, el error	
// aparece en produccion	// Error en compilacion:
	// sumar(5, "3") <- ERROR

■ Adopcion gradual: puedes agregar TypeScript a un proyecto JS existente poco a poco, archivo por archivo. No necesitas reescribir todo desde cero.

3. Instalacion y Configuracion

Instalar TypeScript globalmente:

```
# Instalar TypeScript de forma global
npm install -g typescript

# Verificar la version instalada
tsc --version

# Compilar un archivo
tsc archivo.ts

# Compilar y vigilar cambios (modo watch)
tsc --watch
```

Iniciar un proyecto TypeScript:

```
# Crear carpeta e inicializar npm
mkdir mi-proyecto && cd mi-proyecto
npm init -y

# Instalar TypeScript como dependencia de desarrollo
npm install --save-dev typescript

# Generar tsconfig.json automaticamente
npx tsc --init
```

El archivo tsconfig.json explicado:

El archivo **tsconfig.json** controla como el compilador de TypeScript procesa tu codigo. Aqui estan las opciones mas importantes que debes conocer:

Opcion	Valor tipico	Descripcion
target	"ES2020"	Version de JS a la que se compila. ES2020 es una buena opcion moderna.
module	"commonjs"	Sistema de modulos. Usa "commonjs" para Node, "ESNext" para web moderno.
strict	true	Activa todas las verificaciones estrictas. MUY recomendado desde el inicio.
outDir	"./dist"	Carpeta donde se guardan los archivos JS compilados.
rootDir	"./src"	Carpeta raiz de tus archivos TypeScript fuente.

<code>esModuleInterop</code>	<code>true</code>	Permite importar modulos CommonJS con la sintaxis de ES6.
------------------------------	-------------------	---

4. Tipos Basicos

La anotacion de tipos en TypeScript es opcional y se escribe con dos puntos (:) despues del nombre de la variable o parametro. TypeScript tambien puede **inferir** el tipo automaticamente en muchos casos.

4.1 Tipos primitivos

```
// Tipos primitivos basicos
let nombre: string = 'Cesar';
let edad: number = 28;
let activo: boolean = true;

// TypeScript INFIERE el tipo automaticamente:
let ciudad = 'Lima'; // TypeScript sabe que es string
// ciudad = 42; // ERROR: no puedes asignar number a string

// Undefined y null
let dato: string | null = null; // puede ser string o null
let ref: number | undefined; // puede ser number o undefined
```

4.2 Arrays y Tuplas

```
// Arrays: dos formas equivalentes
let numeros: number[] = [1, 2, 3];
let nombres: Array<string> = ['Ana', 'Luis'];

// Tuplas: array con tipos fijos por posicion
let coordenada: [number, number] = [40.7, -74.0];
let persona: [string, number] = ['Cesar', 28];

// persona = [28, 'Cesar']; // ERROR: orden y tipos importan
```

4.3 Any, Unknown y Never

Estos tres tipos especiales son importantes de entender desde el principio para no caer en malos habitos (especialmente con **any**):

any	Desactiva la verificacion de tipos. Evitalo: es como volver a JS puro.
unknown	Tipo desconocido pero seguro. Debes verificar el tipo antes de usarlo.
never	Tipo de valores que nunca ocurren. Util para funciones que siempre lanzan errores.

4.4 Uniones y Tipos Literales

```
// Union: la variable puede ser uno de varios tipos
let id: string | number;
id = 'abc-123'; // OK
id = 42; // OK
// id = true; // ERROR

// Tipos literales: valores exactos permitidos
let direccion: 'norte' | 'sur' | 'este' | 'oeste';
direccion = 'norte'; // OK
// direccion = 'arriba'; // ERROR

// Util para estados de una app:
type Estado = 'cargando' | 'exito' | 'error';
let estado: Estado = 'cargando';
```

5. Funciones en TypeScript

Tipar funciones es una de las habilidades mas utiles en TypeScript. Debes anotar tanto los parametros como el valor de retorno.

Sintaxis basica:

```
// Funcion con tipos en parametros y retorno
function saludar(nombre: string, edad: number): string {
  return `Hola ${nombre}, tienes ${edad} anyos`;
}

// Funcion flecha tipada
const multiplicar = (a: number, b: number): number => a * b;

// Funciones que no retornan nada usan: void
function log(mensaje: string): void {
  console.log(mensaje);
}
```

5.1 Parametros opcionales y por defecto:

```
// Parametro opcional con ?
function crearUsuario(nombre: string, rol?: string): string {
  return `${nombre} - ${rol ?? 'usuario'}`;
}

crearUsuario('Cesar'); // OK: 'Cesar - usuario'
crearUsuario('Cesar', 'admin'); // OK: 'Cesar - admin'

// Parametro con valor por defecto
function potencia(base: number, exp: number = 2): number {
  return Math.pow(base, exp);
}

potencia(3); // 9 (exp usa el default: 2)
potencia(3, 3); // 27
```

5.2 Tipos para funciones (callbacks):

```
// Definir el tipo de una funcion
type Operacion = (a: number, b: number) => number;

function calcular(a: number, b: number, op: Operacion): number {
  return op(a, b);
}

const suma: Operacion = (a, b) => a + b;
calcular(10, 5, suma); // 15
```

```
calcular(10, 5, (a,b)=>a*b); // 50
```

■ Regla de oro: si una funcion retorna siempre el mismo tipo, anotalo explicitamente. TypeScript puede inferirlo, pero ser explicito mejora la legibilidad del codigo.

6. Interfaces

Las interfaces son la herramienta principal de TypeScript para describir la **forma** de un objeto. Piensa en ellas como un contrato: cualquier objeto que implemente la interface debe tener exactamente las propiedades definidas (y sus tipos).

Ejemplo basico:

```
// Definir una interface
interface Usuario {
  id: number;
  nombre: string;
  email: string;
}

// Usar la interface como tipo
function mostrarUsuario(user: Usuario): void {
  console.log(`${user.id}: ${user.nombre}`);
}

const cesar: Usuario = {
  id: 1,
  nombre: 'Cesar',
  email: 'saavedra0706@gmail.com'
};

mostrarUsuario(cesar); // OK
```

6.1 Propiedades opcionales y readonly:

```
interface Producto {
  readonly id: number; // No se puede modificar despues de crear
  nombre: string;
  precio: number;
  descripcion?: string; // Propiedad opcional
  imagenes?: string[]; // Array opcional
}

const laptop: Producto = { id: 1, nombre: 'Laptop', precio: 999 };
// laptop.id = 2; // ERROR: readonly no permite cambios

// Las propiedades opcionales pueden estar ausentes:
const libro: Producto = { id: 2, nombre: 'Clean Code', precio: 35 };
```

6.2 Extender interfaces:

Las interfaces pueden extenderse (heredar) de otras para reutilizar tipos comunes, similar a la herencia en POO:

```
interface Persona {
  nombre: string;
  edad: number;
}

// Empleado hereda todo de Persona y agrega mas campos
interface Empleado extends Persona {
  empresa: string;
  salario: number;
}

// Incluso se puede extender de multiples interfaces:
interface Gerente extends Empleado {
  equipo: string[];
}

const gerente: Gerente = {
  nombre: 'Ana', edad: 35,
  empresa: 'TechCorp', salario: 8000,
  equipo: ['Luis', 'Maria', 'Pedro']
};
```

6.3 Interfaces vs Type Aliases:

Tanto **interface** como **type** sirven para definir tipos de objetos. La diferencia clave es que las interfaces son extensibles (puedes agregarles propiedades despues de declararlas), mientras que los type aliases son mas versatiles para tipos complejos:

interface	type alias
// Interface	// Type alias
interface Animal {	type Animal = {
nombre: string;	nombre: string;
}	};
// Puedes extenderla despues:	// No puedes redeclararla
interface Animal {	// Pero es mejor para uniones:
sonido: string;	type ID = string number;
}	type Resp = 'ok' 'error';
// Ahora Animal tiene ambas!	

■ Regla practica: usa interface para describir objetos y clases. Usa type para uniones, intersecciones o tipos complejos. En caso de duda, prefiere interface.

7. Clases en TypeScript

TypeScript potencia enormemente las clases de ES6 agregando tipos, modificadores de acceso y otras características propias de lenguajes orientados a objetos como Java o C#.

Clase básica con tipos:

```
class Persona {
  nombre: string;
  edad: number;

  constructor(nombre: string, edad: number) {
    this.nombre = nombre;
    this.edad = edad;
  }

  saludar(): string {
    return `Hola, soy ${this.nombre}`;
  }
}

const p = new Persona('Cesar', 28);
p.saludar(); // 'Hola, soy Cesar'
```

7.1 Modificadores de acceso:

TypeScript agrega tres modificadores que controlan quien puede acceder a cada propiedad o método:

public	Accesible desde cualquier lugar (es el valor por defecto).
private	Solo accesible dentro de la misma clase.
protected	Accesible dentro de la clase y sus subclases (herencia).

```
class CuentaBancaria {
  public titular: string;
  private saldo: number; // Solo accesible internamente
  protected banco: string;

  constructor(titular: string, saldo: number) {
    this.titular = titular;
    this.saldo = saldo;
    this.banco = 'BancoX';
  }
}
```



```

public depositar(monto: number): void {
  if (monto > 0) this.saldo += monto;
}

public getSaldo(): number {
  return this.saldo; // Solo asi se expone el saldo
}
}

const cuenta = new CuentaBancaria('Cesar', 1000);
cuenta.depositar(500);
cuenta.getSaldo(); // 1500
// cuenta.saldo; // ERROR: saldo es private

```

7.2 Shorthand en el constructor:

TypeScript tiene un atajo muy practico: puedes declarar y asignar propiedades directamente en los parametros del constructor:

```

// En lugar de esto (forma larga):
class ProductoLargo {
  nombre: string;
  precio: number;
  constructor(nombre: string, precio: number) {
    this.nombre = nombre;
    this.precio = precio;
  }
}

// Puedes escribir esto (shorthand):
class Producto {
  constructor(
    public nombre: string,
    public precio: number,
    private stock: number = 0
  ) {} // El constructor queda vacio!
}

const p = new Producto('Laptop', 999, 10);
p.nombre; // 'Laptop'
p.precio; // 999

```

7.3 Herencia:

```

class Animal {
  constructor(public nombre: string) {}
}

```

```

mover(distancia: number = 0): void {
  console.log(`${this.nombre} se movio ${distancia}m`);
}
}

class Perro extends Animal {
  ladrar(): void {
    console.log('Guau!');
  }

  // Sobreescibir el metodo padre
  mover(distancia: number = 5): void {
    console.log('Corriendo...');
    super.mover(distancia); // Llama al metodo del padre
  }
}

const perro = new Perro('Rex');
perro.ladrar(); // 'Guau!'
perro.mover(); // 'Corriendo...' + 'Rex se movio 5m'

```

7.4 Clases que implementan interfaces:

Las clases pueden **implementar** interfaces, lo que garantiza que la clase tenga todos los metodos y propiedades requeridos. Es la base de la programacion orientada a contratos en TypeScript:

```

interface Serializable {
  serializar(): string;
  deserializar(data: string): void;
}

class Config implements Serializable {
  constructor(private datos: Record<string, string> = {}) {}

  serializar(): string {
    return JSON.stringify(this.datos);
  }

  deserializar(data: string): void {
    this.datos = JSON.parse(data);
  }
}

```

8. Genericos (Generics)

Los genericos permiten escribir codigo reutilizable que funciona con distintos tipos sin perder la seguridad de tipos. Piensa en ellos como **parametros de tipo**: igual que una funcion acepta parametros de valor, un generico acepta parametros de tipo.

```
// Sin genericos, necesitas usar 'any' (malo):
function primerElemento(arr: any[]): any { return arr[0]; }

// Con genericos, TypeScript infiere el tipo:
function primerElemento<T>(arr: T[]): T {
  return arr[0];
}

const num = primerElemento([1, 2, 3]); // num: number
const str = primerElemento(['a', 'b']); // str: string

// Interface generica
interface Respuesta<T> {
  data: T;
  exito: boolean;
  mensaje: string;
}

// Usar la interface con distintos tipos:
const respUsuario: Respuesta<Usuario> = {
  data: { id: 1, nombre: 'Cesar', email: 'c@c.com' },
  exito: true,
  mensaje: 'Usuario encontrado'
};
```

■ Los genericos son muy comunes en APIs y repositorios de datos. Una vez que los entiendes, los encontraras en todas partes: `Array<T>`, `Promise<T>`, `useState<T>`...

9. Flujo de Trabajo Recomendado

Aquí tienes una guía rápida del camino recomendado para adoptar TypeScript en tus proyectos de forma ordenada y sin frustraciones:

Paso 1	Activa strict mode Asegurate de tener "strict": true en tu tsconfig.json desde el inicio. Evita desactivarlo; los errores que detecta te ahorran horas de debugging.
Paso 2	Empieza por los datos Define interfaces o types para los objetos de tu dominio primero (usuarios, productos, respuestas de API). Esto da estructura a todo lo demás.
Paso 3	Tipa tus funciones Añade tipos a parametros y valores de retorno. Deja que TypeScript infiera los tipos de variables locales simples.
Paso 4	Evita any a toda costa Si TypeScript te pide usar any, considera unknown + type guard en su lugar. Cada any es una deuda técnica que pagas más tarde.
Paso 5	Usa el IDE VS Code con TypeScript es una experiencia excelente: autocompletado, errores en línea, refactoring seguro. Aprovechalo al máximo.

10. Recursos y Siguietes Pasos

Ya tienes las bases para escribir TypeScript productivo. Aqui estan los mejores recursos para continuar tu aprendizaje:

Documentacion oficial typescriptlang.org/docs	La fuente mas completa y actualizada. El Handbook es excelente.
TypeScript Playground typescriptlang.org/play	Editor online para probar codigo TS sin instalar nada.
Total TypeScript (Matt Pocock) totaltypescript.com	Cursos interactivos con ejercicios practicos. Muy recomendado.
Ejercicio practico Migrar un proyecto JS a TS	Toma un proyecto pequeno de JavaScript y migra archivo por archivo. Es la mejor practica.

Ruta de aprendizaje recomendada despues de esta guia

- Decoradores y Metadata (muy usados en NestJS y Angular)
- Utility Types: Partial, Required, Pick, Omit, Record
- Type Guards y Narrowing avanzado
- Modulos y namespaces en proyectos grandes
- TypeScript con frameworks: React (TSX), Node/Express, NestJS

Felicitaciones por completar la guia. El siguiente paso es practicar: abre VS Code, crea un archivo .ts y empieza a tipar tu codigo. Los errores del compilador son tus amigos: te ensenyan el camino correcto.